Relatório executivo EasyShop

Visão geral

O EasyShop implementa cadastro e listagem de produtos em LiteDB e clientes em SQLite, com interface web local e aplicativo Android nativo. A implementação inicial ocorreu em 11/11/2026, com revisão e inclusão de Android em 12/11/2026, sob supervisão do aluno Gabriel Tavares. O prazo acadêmico informado é 30/11/2026. O vídeo permanece sob responsabilidade do colaborador.

Decisões arquiteturais

O workspace estava vazio e não havia SDK .NET disponível no PATH. Foi instalado o SDK 10.0.404 em diretório do usuário, sem alteração do PATH global. A stack escolhida foi C#/.NET 10, ASP.NET Core, HTML/CSS/JavaScript, LiteDB 5.0.21 e Microsoft.Data.Sqlite 10.0.15. O usuário autorizou seguir a melhor opção na ausência de plataforma obrigatória.

A versão inicial utilizou interface web local. Após solicitação explícita do usuário, foi instalado o workload Android 36.1.2 e acrescentado um aplicativo nativo .NET for Android, com APK que inclui x86_64 e arm64. O emulador utilizado é Android 16/API 36 x86_64. Os mesmos arquivos de entidades, regras e repositórios são compilados nas duas plataformas, evitando divergência de validações. Não foi realizado teste em aparelho físico. Caixa, dívidas, frotas, viagens, dashboards, RBAC e sincronização estão não aplicáveis ao escopo EasyShop - pendentes de confirmação.

Na versão web, a UI chama endpoints HTTP; no Android, a Activity chama diretamente os serviços. Em ambas, os serviços aplicam regras e os repositórios fazem persistência. A web utiliza injeção de dependências; a Activity compõe e descarta seus repositórios. SQLite utiliza parâmetros SQL e restrição de unicidade; LiteDB armazena produtos como documentos com decimal, categorias e atributos. Mensagens ao usuário omitem detalhes internos de falhas. O servidor web escuta somente no loopback; o APK não depende dele.

Detalhamento dos módulos

Produtos: cadastro, listagem, preço positivo com duas casas decimais, categorias e atributos opcionais, validação de tamanho e consolidação de categorias repetidas. Produtos de mesmo nome são permitidos. Não há controle de quantidade de estoque.

Clientes: nome e sobrenome obrigatórios, e-mail validado e normalizado, cadastro e listagem. Restrição única no SQLite protege a integridade mesmo com concorrência. Não há envio de e-mail nem verificação de titularidade.

Matriz de rastreabilidade

Requisito	Implementação	Evidência	Status
Produtos LiteDB	ProductService e ProductRepository	Testes de reabertura; produtos.png	Implementado e validado
Clientes SQLite	CustomerService e CustomerRepository	Testes de normalização e persistência; clientes.png	Implementado e validado
Arquitetura em camadas	wwwroot, Services, Data, Models	Código e diagrama README	Implementado
Regras e duplicidade	Serviços e restrição UNIQUE	13 testes xUnit; email-duplicado.png	Validado
Interface e estados	HTML, CSS e app.js	ui-results.json e capturas reais	Validado em navegador automatizado
Build e análise	.NET build e format	build.txt; comandos registrados	Aprovado sem avisos
Empacotamento	Publish Windows x64 autocontido	artifacts/publish/win-x64	Gerado
Documentação	README, manual, relatório, roteiro	Arquivos em docs	Entregue
Repositório público	GitHub, após autorização explícita	Consultar docs/entrega.md	Ver link e estado final no checklist
Convite ao colaborador	GitHub	Usuário não informado	Pendente
Vídeo 3 a 5 minutos	Roteiro de 4 minutos	docs/video/roteiro.md	Roteiro entregue; gravação e link pendentes
Android	Activity nativa e APK com dados locais	evidencias/android e android-build.txt	Verificado no emulador API 36 x86_64
Logística/finanças	Módulos não implementados	Decisão de escopo	Não aplicável ao escopo EasyShop - pendente de confirmação

Validação e evidências

Build web Release e Android Debug aprovados sem avisos nem erros. Os 13 casos xUnit passaram: produtos inválidos, reabertura LiteDB, clientes inválidos, normalização, reabertura SQLite, duplicidade concorrente, normalização de whitespace no nome e rejeição de chave de atributo vazia. O teste Playwright executou cadastro real, listagem, preço zero, duplicidade, estados vazios e viewport web de 390 pixels sem overflow horizontal, sem erros JavaScript. A verificação Android usa ADB/UIAutomator no emulador, com instalação do APK e execução dos dois módulos no sandbox do aplicativo. Resultados e capturas estão em evidencias/android. Essas capturas são reais, obtidas da aplicação com dados fictícios; não são mockups. As sessões foram automatizadas, sem alegação de teste humano ou aparelho físico.

`dotnet format --verify-no-changes --no-restore` aprovado. A auditoria NuGet não reportou pacotes vulneráveis na consulta registrada. Isso representa o estado da fonte de avisos no momento da consulta e não garante ausência de toda vulnerabilidade possível. O registro de restart e a execução do pacote estão em evidencias/restart-results.json.

Infraestrutura e custos

Execução exclusivamente local, sem recursos de nuvem provisionados. Nenhum serviço pago foi contratado nesta execução. O hardware e a conexão já pertencem ao usuário; valores de aquisição, energia e internet estão a confirmar. Não há custo financeiro comprovado informado. Publicação do código utiliza repositório público GitHub autorizado pelo usuário.

Cronograma planejado e executado

Cronograma original de sprints: a confirmar, não fornecido. Prazo planejado informado: 30/11/2026. Em 11/11/2026 foram realizados auditoria do ambiente, instalação do SDK, implementação dos dois módulos, testes automatizados, capturas e documentação. Em 12/11/2026 foram realizados auditoria do briefing, inclusão do alvo Android, geração do APK e verificação em emulador. Gravação, publicação do vídeo e confirmação do enunciado de logística/finanças permanecem pendentes. Não foram inventadas sprints ou horas de esforço.

Dificuldades e lições aprendidas

A ausência inicial do SDK foi resolvida por instalação no perfil do usuário. A primeira restauração detectou aviso de vulnerabilidade em SQLitePCLRaw 2.1.11; a atualização de Microsoft.Data.Sqlite trouxe a versão transitiva 2.1.13 e a consulta de vulnerabilidades deixou de apresentar alertas. O teste de interface exigiu ajuste do seletor do campo de e-mail para incluir a marca visual de obrigatório; houve também correção de codificação do script de teste após leitura pelo PowerShell. Esses ajustes foram seguidos de nova execução.

A separação das regras de negócio permitiu testes com bancos isolados, sem depender da UI. A unicidade deve ser garantida pelo banco além da validação de formulário. Evidências reproduzíveis e identificação clara de pendências evitam confundir protótipo demonstrável com entrega acadêmica final.

Na revisão Android, o teclado numérico padrão do emulador removia a vírgula em 29,90. O campo foi corrigido para aceitar explicitamente vírgula e ponto, e o teste passou a conferir o texto digitado antes do cadastro. A primeira execução de CI Android falhou por tentar instalar o pacote obsoleto tools; o workflow passou a solicitar platform-tools explicitamente. A verificação local do APK incluiu reinício em modo avião e leitura em resolução compacta.

Limitações e próximos passos

Sem edição, exclusão, login, criptografia de banco, paginação, sincronização ou acesso em rede. Adequado à demonstração acadêmica local, não a serviço multiusuário público. O APK usa assinatura de desenvolvimento; não houve teste físico ou publicação em loja. Compatibilidade comprovada somente no ambiente de emulação registrado; outros aparelhos e versões exigem teste adicional. Para concluir a submissão, informar colaborador, gravar e publicar vídeo, verificar acesso aos links e validar com o professor se há outro enunciado aplicável.